# Architecture and Implementation Blueprint: Cross-Browser Orchestration of HPC Grid Workers via LLM Interface Interception

## Executive Summary

The orchestration of high-performance computing (HPC) workloads traditionally requires specialized knowledge of secure shell (SSH) protocols, workload managers such as Slurm, and containerization runtimes like Apptainer. Conversely, modern large language model (LLM) interfaces, most notably the official Anthropic Claude web application, offer highly optimized, multimodal user experiences but remain fundamentally disconnected from localized, secure academic data enclaves. Bridging this gap by seamlessly intercepting prompts from a production LLM web interface and redirecting them to a generalized university HPC grid represents a paradigm shift in computational research environments.

This comprehensive analysis confirms the absolute technical feasibility of this architecture. By leveraging a Manifest V3 cross-browser extension, Open OnDemand (OOD) reverse proxies, and a generalized containerized backend, it is possible to hijack the native web interface of consumer LLMs. This architecture allows researchers to leverage the pristine, production-grade user interface of the web application while redirecting the actual prompt payloads to an isolated HPC compute node. Crucially, this system is not hardcoded to a single institution's infrastructure. An abstraction layer can be implemented utilizing a functional VS Code extension blueprint—originally developed for Wayne State University—that utilizes an LLM to parse generic institutional documentation. This enables the dynamic generation of configuration schemas, thereby standardizing the deployment across any academic grid regardless of its specific architectural idiosyncrasies.

The following report details the specific implementation mechanisms required to achieve this universal deployment, utilizing browser-agnostic programming frameworks, robust message-passing architectures, generalized HPC containerization protocols, and deterministic LLM prompt validation methodologies.

## The Browser-Agnostic Extension Framework

To achieve true browser agnosticism and deploy the extension seamlessly across Google Chrome, Apple Safari, Mozilla Firefox, and Microsoft Edge, the architecture must abandon native, browser-specific extension APIs in favor of a unified build framework. The fragmentation of the browser extension ecosystem, exacerbated by the transition from Manifest V2 to Manifest V3, necessitates a robust compilation pipeline capable of handling differing security paradigms and namespace conventions.

### Evaluating Cross-Browser Build Engines

The web extension development landscape features several modern frameworks designed to abstract the complexities of cross-browser compatibility. Frameworks such as Plasmo, Extension.js, and CRXJS have emerged as popular alternatives to vanilla JavaScript development[1]. However, when evaluating the specific requirements of a robust, widely deployable academic tool, the Web Extension Toolkit (WXT) framework serves as the optimal foundation[2].

Built on top of the Vite build engine, WXT allows developers to maintain a single TypeScript-based codebase that dynamically compiles into both Manifest V2 and Manifest V3 packages depending on the target browser[2]. This is a critical capability, as browsers like Firefox and Safari have maintained robust support for Manifest V2, recognizing that Manifest V3 imposes severe limitations on certain background processes, while Chrome and Edge rigidly enforce Manifest V3[4]. WXT streamlines development through file-based routing and auto-imports, eliminating the boilerplate associated with manual manifest generation[3]. Instead of manually configuring entry points, the framework automatically infers the manifest based on the project structure[3].

Alternative frameworks, while powerful, present limitations in a truly browser-agnostic context. Plasmo offers exceptional React integration and is widely used for Chrome Manifest V3 extensions, but its official support documentation lists Firefox Manifest V3 as experimental and requires manual wrappers for Safari targets[8]. Extension.js provides a zero-configuration approach, but relies on compiling whatever is explicitly declared in a pre-existing manifest rather than generating it dynamically from file structures[1]. CRXJS operates as a Vite plugin, offering maximum control, but is highly susceptible to breakage during standard Vite bundler upgrades and lacks the high-level conveniences of a full framework[1]. WXT natively targets all major environments out of the box, handling browser-specific manifest differences and providing unified APIs[7].

| Framework | Build Engine | Cross-Browser Output Capability | Manifest Generation Strategy | Primary Architecture |
|---|---|---|---|---|
| **WXT** | Vite | First-class Chrome, Firefox, Safari, Edge | Automatic via Config/Files | File-based Entrypoints |
| **Plasmo** | Parcel-based (Custom) | Chrome/Edge (Firefox/Safari require manual shims) | Automatic via Config | File-based Entrypoints |
| **Extension.js** | Webpack/Vite | Chrome, | Direct | Config-based |

| | | | | |
|---|---|---|---|---|
| | | Firefox, Edge | Read/Compile | |
| **CRXJS** | Vite | Chrome/Edge (Brittle on upgrades) | Direct Read/Compile | Config-based |

### Bridging Namespace Disparities via Polyfills

Beyond manifest generation, the actual JavaScript execution environment differs significantly across browser vendors. Firefox utilizes a browser namespace with Promise-based APIs by default[11]. Conversely, Chrome has historically utilized callback-based APIs under the chrome namespace, although modern versions are incrementally supporting Promises[11]. Edge largely follows Chrome's conventions due to its Chromium underlying engine[11].

Attempting to manage these disparities through conditional logic results in brittle, difficult-to-maintain codebases. WXT inherently simplifies this by providing wrappers for different browser storage APIs, allowing a single line of code to interact with Chromium, Safari, and Gecko engines seamlessly[12]. Furthermore, the architecture heavily relies on the mozilla/webextension-polyfill library. This library augments the WebExtensions API methods, ensuring that all cross-browser extension interactions are promise-based and operate under a unified namespace[13]. By isolating browser-specific network interception, notification handling, and background worker configuration behind this abstraction layer, the core business logic remains pristine and universally executable[11].

## Manifest V3 Execution Worlds and DOM Interception

The core mechanism of this orchestration system relies on capturing the user's prompt before it is transmitted to Anthropic's cloud servers. Modern web applications utilize the window.fetch API or XMLHttpRequest for outbound network requests. Intercepting these requests requires injecting a JavaScript payload into the execution context of the target web application. This presents a unique challenge under the strict architectural constraints of Manifest V3.

### Navigating the Isolated and Main Worlds

In Manifest V3, content scripts run in an ISOLATED world by default. This architectural decision dictates that while the content script shares the Document Object Model (DOM) with the host page, it maintains a completely separate JavaScript execution environment. Consequently, monkey-patching window.fetch within an ISOLATED world will only intercept fetch calls made by the extension itself; it will not affect or even observe the fetch calls executed by the primary web application running in the MAIN world.

To overcome this fundamental limitation, the extension must explicitly declare a content script destined for the MAIN execution world. Recent updates to Chrome and standard Manifest V3 specifications allow extensions to inject scripts directly into the MAIN world using the world: "MAIN" configuration directive[15]. Alternatively, this can be achieved by programmatically appending a <script> tag to the DOM, which forces the browser to execute the script within the

host page's context.
Once successfully injected into the MAIN world, the script executes a precise sequence of operations to hijack the data pipeline. Initially, it stores a reference to the original, native window.fetch function to ensure that unrelated network traffic can be processed normally. It then overwrites the global window.fetch variable with a custom asynchronous proxy function. Every outbound network request is subsequently funneled through this proxy. The proxy function inspects the request URL; if the URL matches the signature of a conversational submission endpoint—such as routes containing /api/organizations/ or /chat_conversations/—the interception logic is triggered[5].
The proxy function then parses the intercepted Request body, extracting the text prompt and any associated multimodal data, such as Base64-encoded images or documents[17]. The outbound request to the external commercial cloud server is actively aborted. Crucially, to prevent the front-end user interface from crashing or displaying a fatal network timeout error, the interceptor immediately constructs and returns a mock Response object. This mocked response simulates a successful API transaction, returning a generic status message indicating that the prompt is being routed to the generalized HPC grid worker, thereby maintaining UI stability[17].

### Secure Cross-Origin Message Transport Architecture

Once the payload is successfully extracted within the MAIN world, it must be transmitted to the HPC grid. Because of strict Cross-Origin Resource Sharing (CORS) and Content Security Policy (CSP) restrictions enforced by production web applications, the injected MAIN world script cannot communicate directly with the HPC endpoint[15].
To circumvent this, the architecture requires a sophisticated, multi-hop asynchronous message bus:

1. **DOM Broadcast:** The MAIN world script utilizes window.postMessage() to broadcast the extracted JSON payload across the DOM.
2. **Isolated Context Reception:** An ISOLATED world content script, operating concurrently, listens for this specific message event.
3. **Extension Background Routing:** The ISOLATED script leverages browser.runtime.sendMessage()—utilizing the aforementioned polyfills for cross-browser promise support—to relay the payload to the extension's Background Service Worker[13].
4. **External Grid Transmission:** The Background Service Worker, operating with elevated network privileges defined in the manifest.json, initiates an HTTP POST request to the HPC grid's reverse proxy endpoint, successfully bridging the gap between the browser and the compute cluster[17].

## Abstracting the HPC Grid Gateway via Open OnDemand

The primary impediment to deploying this architecture universally lies in the extreme heterogeneity of academic university grid configurations. Different computational centers utilize varying login node architectures, workload managers, and deeply entrenched firewall

policies. Abstracting these specificities is paramount. The system achieves this by relying on Open OnDemand (OOD), an open-source HPC web portal utilized by hundreds of research computing centers globally to provide web-based access to supercomputing resources.

## The Reverse Proxy Routing Mechanisms

OOD features a highly capable built-in reverse proxy that securely tunnels HTTP and WebSocket traffic from the public internet, through the university firewall, directly to an isolated, active compute node[18]. This bypasses the need for students to configure complex SSH tunnels, manage VPN port forwarding, or understand terminal-based networking.

The browser extension must dynamically detect and route traffic through one of two primary OOD proxy configurations: absolute proxying (node) or relative proxying (rnode).

- **Absolute Proxying (node URI):** A request structured as /node/<host>/<port>/<path> reverse proxies the traffic to the backend server utilizing the full, untouched URL path[18]. For instance, a request to /node/compute01/8080/api/run is sent to the backend server exactly as /node/compute01/8080/api/run. This requires the backend Python application to be explicitly aware of the sub-URI prefix, often requiring dynamic base URL rewriting, which complicates deployment across different universities[19].
- **Relative Proxying (rnode URI):** A request structured as /rnode/<host>/<port>/<path> strips the proxy identifier and host information entirely, sending only the relative path to the backend[18]. A request to /rnode/compute01/8080/api/run arrives at the backend simply as /api/run. This is highly preferable for generalized deployments, as the underlying Python web server can operate seamlessly without needing to dynamically rewrite its base URL or route handlers[20].

| Proxy Configuration | OOD URI Structure | Path Received by Backend Web Server | Architectural Ideal Use Case |
|---|---|---|---|
| **Node (Absolute)** | /node/compute01/8080/api/run | /node/compute01/8080/api/run | Applications explicitly configured with dynamic sub-URI base URLs. |
| **Rnode (Relative)** | /rnode/compute01/8080/api/run | /api/run | Frameworks expecting root-level relative paths (preferred for generalized APIs). |

OOD administrators strictly govern reverse proxy targets using a host_regex configuration within the ood_portal.yml configuration file, which typically restricts proxying to internal

compute node domains for security[18]. The extension's background worker is designed to interrogate the active browser tabs, identify the active OOD session, and extract the dynamically assigned compute <host> and <port> from the URL string, ensuring that the payload is routed accurately to the specific user's allocated job[17].

### Zero-Credential Browser Authentication

Traditional remote execution pipelines rely on students generating SSH keys or API tokens and managing localized configuration files. This introduces significant user friction and exposes security risks if keys are mishandled[22]. Because the entire application flow relies on the Open OnDemand reverse proxy, authentication is handled natively by the browser's existing session state[17].

When a user authenticates with their university's Single Sign-On (SSO) or Shibboleth provider to access the OOD dashboard, the browser generates secure, HttpOnly session cookies. When the extension's Background Service Worker transmits a payload to the /rnode/ endpoint, it includes the credentials: 'include' directive in the fetch request. The browser automatically attaches the active OOD session cookies to the outbound request. The OOD proxy validates the cookie, confirms the user is authorized to access the specific backend compute node, and forwards the traffic. At no point does the browser extension or the backend ever read, copy, or store a password, OAuth token, or SSH key[17].

## The Translation Protocol: Generalizing the Wayne State Blueprint

The original premise assumes the existence of a functional VS Code extension, developed specifically for the Wayne State University grid, that handles the intricacies of remote execution[17]. To achieve true browser-agnostic and institution-agnostic deployment, the highly specific protocols of the "Ashley" Wayne State extension must be abstracted. The architecture dictates that this translation is facilitated dynamically by an LLM parsing the target university's technical documentation.

### Analyzing the Baseline Protocol

The "Ashley" VS Code extension serves as the foundational blueprint. According to its architectural documentation, it operates by managing SSH configuration blocks, generating ed25519 key pairs, and establishing persistent connections to the login node (e.g., warrior)[22]. It submits jobs using sbatch, monitors them via squeue, and writes requests to a shared CephFS filesystem utilizing a polling inbox/outbox JSON messaging queue[22]. Crucially, it manages an Apptainer sandbox built from a Node 22 base image, binds host paths (like /tmp to /dev/shm), and handles Slurm executable wrappers to allow the containerized LLM worker to submit subsequent jobs[22].

While highly effective, this protocol is deeply entangled with Wayne State specifics, such as the voh3.nas.grid.wayne.edu proxy jump, the CentOS 7 (glibc 2.17) host OS requiring compatibility libraries, and the exact paths for the lab's shared volume (/rs/rs_grp_oschome/shared/ashley/)[22].

### Dynamic Institution Configuration Schema Generation

To generalize this for any university cluster, the user uploads their specific institution's HPC

documentation (e.g., wiki pages, PDF user guides, Slurm configuration parameters) to a local instance of Claude via the VS Code extension interface. The LLM is instructed to map the institution's specific idiosyncrasies against a generalized template.
This process outputs a standardized Dynamic Institution Configuration Schema (JSON format) containing:

1. **Network Topology:** Extracts the specific OOD domain (e.g., ondemand.hpc.university.edu), the login node hostnames, and any required SSH jump proxies.
2. **Workload Management:** Identifies standard CPU, GPU, and requeue/preemptible Slurm partition names, adjusting the worker.sbatch.tpl job template accordingly.
3. **Storage Architecture:** Maps the high-speed scratch space paths (e.g., /scratch/$USER or $BIGWORK), the read-only dataset mounts, and identifies the underlying file system type (e.g., Lustre, GPFS) to determine container caching compatibility[23].
4. **Containerization Runtime:** Determines if the cluster utilizes Apptainer, Singularity, or Podman, and explicitly checks the documentation for --fakeroot or unprivileged user namespace support[25].

By decoupling the hardcoded domain names and filesystem paths into this universally parseable JSON state object, the browser extension and the backend execution engine can dynamically configure the execution environment, rendering the original Wayne State protocols universally applicable.

# Compute Node Containerization and Unprivileged Execution

Upon arrival at the OOD reverse proxy, the HTTP payload is routed to a lightweight Python FastAPI server running on the user's allocated compute node[17]. This server acts as the primary orchestrator between the incoming web requests and the persistent LLM worker operating within a container. Deploying containerized workloads in HPC environments requires navigating strict security policies that differ drastically from standard enterprise Docker deployments.

## Apptainer and the Fakeroot Paradigm

HPC grids strictly rely on Apptainer (formerly Singularity) due to stringent security requirements regarding root-level privileges[25]. Apptainer was created to bring containers to HPC environments, eventually being adopted by the Linux Foundation to ensure it remained free of corporate control[25]. Apptainer operates under an "integration over isolation" philosophy, meaning the user inside the container is identical to the user outside the container[25]. By default, it does not manage cgroups or enter new PID, network, or IPC namespaces, ensuring that containerized processes communicate seamlessly with the host's batch scheduling system[25].
The generalized architecture must account for varying levels of administrative restrictions across universities. The Wayne State blueprint indicates the use of the --fakeroot flag[22]. When Apptainer is run with --fakeroot, it creates a user namespace for the container, mapping UIDs and GIDs in that namespace to different host IDs[23]. This simulates root privileges inside the container, allowing the LLM worker to utilize standard package managers (like apt or npm)

during runtime.
However, many legacy or highly secured university clusters disable unprivileged user namespaces at the kernel level or restrict the use of setuid binaries[26]. If the Dynamic Institution Configuration Schema detects that --fakeroot is unsupported, the system must fallback gracefully. In these environments, the SIF (Singularity Image Format) container image cannot be modified at runtime. Instead, the container must be extracted to a standard sandbox directory to execute, or the user must rely strictly on Python virtual environments mounted externally, sacrificing some execution speed for universal compatibility[26].

## OverlayFS and High-Performance I/O

HPC workloads, particularly data science pipelines written by LLMs, require significant writable scratch space for tensor caching and intermediate data processing. Apptainer utilizes the kernel's OverlayFS to provide a writable layer on top of a read-only SIF image[23].
If the host environment permits, the generalized script dynamically provisions a sparse EXT3 overlay image utilizing the apptainer overlay create command, placing it in the user's high-speed scratch directory[24]. When the container is launched with the --overlay flag, it unlocks write capabilities across the internal system paths, allowing the LLM worker to natively execute pip installations or write software directly into /opt paths without modifying the immutable shared image[24].
Crucially, the translation protocol must understand underlying storage limitations. Network file systems common in HPC, such as NFS, Lustre, or GPFS, frequently do not support UID/GID mapping within user namespaces[23]. Consequently, placing writable overlays or cache directories (SINGULARITY_CACHEDIR) on these network mounts while utilizing --fakeroot will result in catastrophic "permission denied" errors[23]. The generalized backend logic rigorously verifies the storage architecture, defaulting to localized /tmp or specifically designated parallel storage when user namespaces are enforced[23].

| Execution Mechanism | Required Capabilities | Advantages | HPC Limitations |
|---|---|---|---|
| **--fakeroot Execution** | Unprivileged user namespaces, UID mapping | Allows package installation, mimics root access | Fails on NFS/Lustre storage; often disabled by HPC admins[23]. |
| **OverlayFS Ext3 Image** | Kernel OverlayFS support | Persistent writable layer, high metadata performance | Fixed file size; writes fail once the image is full[24]. |
| **Sandbox Directory** | Standard file | Extremely | Slow startup times; |

| | | | |
|---|---|---|---|
| | system permissions | universally compatible; easy to debug | high metadata load on parallel file systems[26]. |

# Deterministic LLM Execution and Structured Output Validation

Once the prompt and associated multimodal files (e.g., PDFs, DICOMs) are successfully staged on the compute node, the system must ensure that the LLM acts deterministically. In a conversational web interface, models tend to generate extensive conversational padding. In an automated HPC environment, this padding causes catastrophic parsing failures when the system expects executable Slurm commands or pipeline automation scripts. To enforce strict behavioral protocols, the architecture leverages context injection coupled with advanced few-shot prompting and rigorous schema validation[17].

## Few-Shot Boundary Schemas and Chain-of-Thought

When the extension intercepts the prompt, it invisibly prepends a highly structured system instruction outlining the specific capabilities of the target HPC grid (as defined by the generated schema). To prevent the LLM from hallucinating commands or dropping required JSON keys, the prompt utilizes dual-example few-shot schemas[31].

Zero-shot prompting forces the model to deduce the shape and tone of the output solely from natural language guidelines, which frequently fails under the weight of production edge cases[31]. High-signal few-shot schemas provide concrete input and output exemplars. Rather than merely providing positive examples, the prompt injection explicitly demonstrates negative boundaries and edge case handling[31]. By providing examples of adversarial inputs or ambiguous requests, the LLM is implicitly trained on the exact structural interaction pattern required[31].

Furthermore, to maximize execution accuracy, the injected schema requires the LLM to output a reasoning_steps array before outputting the final executable command. LLMs generate tokens autoregressively; each token depends on the tokens that preceded it. If a model is forced to produce a final execution command immediately, it must commit to an answer before performing any analytical computation[31]. By forcing the model to calculate its analytical steps in a designated scratchpad array within the JSON output, the final classification tokens are conditioned on that structured analysis, significantly improving the reliability of the generated Slurm scripts or data pipelines[31].

## Schema Contracts and Autonomous Error Correction

Once the LLM outputs the structured payload, the FastAPI backend must safely parse and validate the data before executing any arbitrary code on the cluster. Treating LLM output as a strongly typed REST API response is a fundamental architectural requirement[33]. The system utilizes validation libraries, such as Zod (or Pydantic in Python), to enforce a rigid schema contract[30]. The schema acts as the single source of truth, deriving both the validation rules and the underlying TypeScript/Python types automatically, preventing the contract from

diverging[34].
The parsing logic implements highly defensive strategies to extract the JSON payload. LLMs consistently wrap outputs in markdown code fences or prepend conversational pleasantries, making basic JSON parsing impossible[30]. The extraction functions strip these fences and utilize robust substring matching to locate the outermost JSON objects[30].
Crucially, the system employs the safeParse methodology rather than standard parse methods[30]. If an LLM hallucinates an enum value—for example, attempting to submit a job to a super-gpu partition that does not exist in the university's configuration—standard parsing throws a fatal syntax error with internal formatting that is useless for programmatic debugging[30]. By utilizing safeParse, the system catches the structural violation gracefully. When a validation failure occurs, it is not treated as an application crash, but rather as an expected branch of the operational flow[34]. The precise error message—generated directly by the schema validator (e.g., "partition must be one of: cpu-standard, gpu-a100, high-mem")—is automatically fed back into the LLM as a subsequent prompt[30]. LLMs demonstrate exceptionally high rates of self-correction when provided with exact Zod validation failures, as the explicit error string provides the necessary semantic context for the model to revise its output[30]. This feedback loop operates entirely autonomously on the compute node. The user operating the claude.ai frontend remains completely shielded from these underlying syntax errors and validation loops, receiving only the final, validated execution logs and scientific analysis streamed back to their browser.

# Conclusion

The cross-browser orchestration of generalized HPC grid workers via LLM web interface interception represents a highly scalable and technically rigorous architecture for modern academic research. By utilizing the WXT framework, the extension achieves true browser agnosticism, deploying seamlessly across Manifest V3 environments while natively navigating the complex execution worlds required for DOM interception.
The integration of Open OnDemand's relative reverse proxy routing completely eliminates the need for complex network tunneling, allowing intercepted HTTP payloads to flow securely from the browser DOM directly to isolated compute nodes using native session cookies. By abstracting institutional idiosyncrasies into a Dynamic Institution Configuration Schema—generated by uploading university documentation to a foundational VS Code extension—the system can be universally deployed without hardcoding infrastructure logic. Furthermore, the combination of Apptainer's unprivileged execution models, dual-example few-shot prompting, and robust schema validation ensures that LLM outputs remain strictly deterministic, secure, and accurate. This architecture successfully decouples the cognitive reasoning capabilities of commercial LLMs from their cloud-hosted constraints, projecting their analytical power directly into localized, high-performance data enclaves while maintaining the pristine user experience of modern web applications.

### Works cited

1. Browser extension framework comparison, https://extension.js.org/docs/compare

2. WXT: Next-gen Web Extension Framework, https://wxt.dev/
3. What Is a Browser Extension? Build Your First One with WXT, https://redreamality.com/blog/browser-extension-development/
4. Targeting Different Browsers - WXT, https://wxt.dev/guide/essentials/target-different-browsers
5. Browser Extension Dev - 02. Using WXT - rxliuli, https://rxliuli.com/blog/browser-extension-dev-02-using-wxt/
6. Manifest V3: The Ghostery Perspective, https://www.ghostery.com/blog/manifest-v3-the-ghostery-perspective
7. Best Practices, WXT, AI Coding Agents, and Future-Proof Strategies, https://www.reddit.com/r/AgentContext_dev/comments/1uddegh/mastering_modern_browser_extension_development_in/
8. Plasmo vs WXT vs CRXJS vs Extension.js: 2026 Comparison, https://merginit.com/blog/23082026-plasmo-vs-wxt-vs-crxjs-extension-js
9. Anouk vs Plasmo — honest comparison, https://anouk.skelfresearch.com/compare/plasmo/
10. The cross-browser extension framework. - GitHub, https://github.com/cezaraugusto/extension-create
11. Cross-Browser Extension Development: Chrome vs. Firefox vs. Edge, https://metadesignsolutions.com/blog/cross-browser-extension-development-chrome-vs-firefox-vs-edge
12. Creating a Browser Extension using SolidJS + WXT II, https://dev.to/devshogun/creating-a-browser-extension-using-solidjs-wxt-ii-4lf5
13. chrome.runtime.onMessage response with async await, https://stackoverflow.com/questions/44056271/chrome-runtime-onmessage-response-with-async-await
14. Building Browser Extensions - pdfcoffee.com, https://pdfcoffee.com/building-browser-extensions-pdf-free.html
15. content_security_policy - Mozilla - MDN Web Docs, https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/manifest.json/content_security_policy
16. What's new in Chrome extensions - Chrome for Developers, https://developer.chrome.com/docs/extensions/whats-new
17. Protocols for Reproducible Machine Learning Research.pdf
18. ood_portal.yml — Open OnDemand 4.2.0 documentation, https://osc.github.io/ood-documentation/latest/reference/files/ood-portal-yml.html
19. Connection View — Open OnDemand 4.2.0 documentation, https://osc.github.io/ood-documentation/latest/how-tos/app-development/interactive/view.html
20. Reverse proxy without Location header modification - Get Help, https://discourse.openondemand.org/t/reverse-proxy-without-location-header-modification/3181
21. Reverse proxy pass multiple sites with similar content structure, https://discourse.openondemand.org/t/reverse-proxy-pass-multiple-sites-with-si

milar-content-structure/1961
22. manual.pdf
23. Installing Singularity — Singularity Admin Guide 3.7 documentation, https://apptainer.org/admin-docs/3.7/installation.html
24. guide:apptainer [Cluster Docs], https://docs.cluster.uni-hannover.de/doku.php/guide/apptainer
25. Apptainer: containers for performance-intensive computing - CIQ, https://ciq.com/products/apptainer
26. Security in Apptainer — Apptainer Admin Guide 1.0 documentation, https://apptainer.org/docs/admin/1.0/security.html
27. Container with Apptainer - BayernCollab, https://collab.dvb.bayern/spaces/UniARZHPCKB/pages/1014857861/Container+with+Apptainer
28. Apptainer Configuration Files, https://apptainer.org/docs/admin/1.3/configfiles.html
29. Apptainer Configuration Files, https://apptainer.org/docs/admin/1.0/configfiles.html
30. Zod + LLMs: How to Validate AI Responses Without Losing Your Mind, https://dev.to/pavelespitia/zod-llms-how-to-validate-ai-responses-without-losing-your-mind-4c5j
31. Production Prompt Engineering Patterns: Few-Shot Schemas, Chain, https://dropoutdeveloper.in/prompt-engineering-examples/
32. Reliable LLM JSON Output: Few-Shot Prompting & Robust Parsing, https://dev.to/maanu07/reliable-llm-json-output-few-shot-prompting-robust-parsing-2f11
33. Type-Safe LLM Outputs with Zod and JSON | Pritam Ghosh, https://mepritam.dev/references/constraining-llms-structured-outputs-zod-jsonschema/
34. Handling LLM Output Safely in Production: A Four-Layer Approach, https://medium.com/@abdullahzengin/handling-llm-output-safely-in-production-a-four-layer-approach-from-schema-to-metrics-f16253118932